

Python Installation & Environment Setup



Suman

16 Jan, 2026 At 7:30 PM

Pre-Reads

Trimester-1

Recommended

Resource



Associated Lectures (1)



Description



Discussions

Python Fundamentals PRE-READ

Program: AIM101 — Artificial Intelligence & Machine Learning Foundation **Pre-read Status:**
MANDATORY — Must be completed before live session

What You Will Learn Today

By the end of Session 1.1, you will be able to:

Explain why Python dominates the AI/ML industry

Create variables and understand how Python stores data

Identify the four fundamental data types (int, float, str, bool)

Convert between data types safely

Apply arithmetic, comparison, and logical operators

Manipulate strings using indexing, slicing, and methods

Build interactive programs using print() and input()

Complete a mini-project: Interactive Greeting Generator

The Business Story: A \$327.6 Million Bug

September 23, 1999 — NASA's Worst Day

The Mars Climate Orbiter had traveled for 9 months across 416 million miles of space. Mission cost: \$327.6 million. Everything was perfect — until it wasn't.

The spacecraft was supposed to orbit Mars at 150 kilometers altitude. Instead, it entered at just 57 kilometers and burned up in the Martian atmosphere. Gone. Instantly.

What Went Wrong?

One software team (Lockheed Martin) wrote code using **imperial units** (pound-force-seconds). Another team (NASA JPL) assumed the data was in **metric units** (Newton-seconds).

The conversion factor between these units? **4.44822**

Nobody checked. Nobody converted. The software happily passed numbers back and forth, but those numbers meant completely different things to each system.

Why This Matters to You

In Python, the number `5` and the text `"5"` look identical to humans. But to Python, they're completely different:

```
# These are NOT the same!  
5 + 3      # Result: 8 (math)  
"5" + "3"  # Result: "53" (text concatenation)
```

Today you'll learn about data types — the difference between a successful program and a \$327 million disaster.

Setup Requirements

CRITICAL: Complete ALL steps below BEFORE the live session

The live session assumes your environment is ready. We will NOT spend class time on installation. If you encounter issues, contact support BEFORE class.

Part 1: Python 3.12 Installation

For Windows Users

Step 1: Download Python 3.12

Open your browser and go to: <https://www.python.org/downloads/>

Click the yellow "Download Python 3.12.x" button

Save the installer file

Step 2: Run the Installer

Double-click the downloaded file

CRITICAL CHECKBOX: At the bottom of the first screen, CHECK the box that says:

☒ **"Add python.exe to PATH"**

This is the most common mistake. If you miss this, Python won't work from the command line.

Click "Install Now"

Wait for installation to complete

Click "Close"

Step 3: Verify Installation

Press `Windows + R` to open Run dialog

Type `cmd` and press Enter

In the black command window, type:

```
python --version
```

You should see: `Python 3.12.x`

Troubleshooting Windows:

Problem	Solution
"python is not recognized"	Reinstall Python and CHECK the "Add to PATH" box
Shows Python 2.x or 3.10	Uninstall old versions, reinstall 3.12
Microsoft Store opens	Disable Python app aliases in Settings > Apps > App execution aliases

For macOS Users

Step 1: Download Python 3.12

Open your browser and go to: <https://www.python.org/downloads/>

Click "Download Python 3.12.x"

Open the downloaded `.pkg` file

Step 2: Run the Installer

Follow the installation wizard

Click "Continue" through the screens

Click "Install"

Enter your Mac password when prompted

Click "Close" when done

Step 3: Verify Installation

Open Terminal (press `Cmd + Space`, type "Terminal", press Enter)

Type:

```
python3 --version
```

You should see: `Python 3.12.x`

Important for Mac: Use `python3` (not `python`) as macOS has a system Python 2.

Troubleshooting macOS:

Problem	Solution
"command not found"	Restart Terminal and try again
Shows wrong version	Check with <code>which python3</code> to see path
Permission denied	Run installer again with admin rights

For Linux Users (Ubuntu/Debian)

Step 1: Update Package List

```
sudo apt update
```

Step 2: Install Dependencies

```
sudo apt install software-properties-common -y
```

Step 3: Add deadsnakes PPA

```
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update
```

Step 4: Install Python 3.12

```
sudo apt install python3.12 python3.12-venv python3.12-dev -y
```

Step 5: Verify Installation

```
python3.12 --version
```

You should see: `Python 3.12.x`

Part 2: Miniconda Installation

Miniconda is a lightweight version of Anaconda that lets us create isolated Python environments. This is industry standard practice.

Why Use Miniconda?

Think of it like having separate toolboxes for different projects:

- Project A needs Tool version 1.0
- Project B needs Tool version 2.0
- Without environments, they conflict!
- With environments, each project has its own isolated toolbox

For Windows Users

Step 1: Download Miniconda

Go to: <https://docs.conda.io/en/latest/miniconda.html>

Download: "Miniconda3 Windows 64-bit"

Save the `.exe` file

Step 2: Install Miniconda

Run the downloaded installer

Click "Next"

Accept the license agreement

Choose "Just Me" (recommended)

Keep the default installation path

IMPORTANT: Check BOTH boxes:

- ☒ Create start menu shortcuts
- ☒ Add Miniconda3 to my PATH environment variable

Click "Install"

Click "Finish"

Step 3: Verify Miniconda

Close any open command windows

Open a NEW command prompt (Windows + R, type `cmd`)

Type:

```
conda --version
```

You should see: `conda 23.x.x` or similar

For macOS Users

Step 1: Download Miniconda

Go to: <https://docs.conda.io/en/latest/miniconda.html>

Download: "Miniconda3 macOS Apple M1 64-bit pkg" (for M1/M2 Macs) OR "Miniconda3 macOS Intel x86 64-bit pkg" (for Intel Macs)

Step 2: Install Miniconda

- Open the downloaded `.pkg` file
- Follow the installation wizard
- Use default settings
- Complete installation

Step 3: Verify Miniconda

- Close Terminal if open
- Open a NEW Terminal window
- Type:

```
conda --version
```

You should see: `conda 23.x.x` or similar

For Linux Users

Step 1: Download Miniconda

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

Step 2: Install Miniconda

```
bash Miniconda3-latest-Linux-x86_64.sh
```

- Press Enter to review license
- Type `yes` to accept
- Press Enter for default location
- Type `yes` to initialize conda

Step 3: Activate and Verify

```
source ~/.bashrc  
conda --version
```

Part 3: Create Your ML Environment

Now we'll create a dedicated environment for this course with Python 3.12.

Step 1: Create the Environment

Open your terminal/command prompt and run:

```
conda create -n mlenv python=3.12 -y
```

This creates an environment named `mlenv` with Python 3.12.

Expected output: You'll see conda downloading packages and a success message.

Step 2: Activate the Environment

```
conda activate mlenv
```

Your prompt should change to show `(mlenv)` at the beginning:

```
(mlenv) C:\Users\YourName>      # Windows  
(mlenv) username@computer:~$    # Mac/Linux
```

Step 3: Install Jupyter Notebook

With your environment activated, run:

```
conda install jupyter -y
```

Wait for the installation to complete.

Step 4: Verify Everything

Run these commands one by one:

```
python --version
```

Expected: `Python 3.12.x`

```
jupyter --version
```

Expected: Multiple version numbers for jupyter components

Part 4: Launch Jupyter Notebook

Starting Jupyter

Make sure your environment is activated: `conda activate mlenv`

Navigate to where you want to save notebooks:

```
cd Documents
mkdir AIM101
cd AIM101
```

Launch Jupyter:

```
jupyter notebook
```

Your default browser will open with the Jupyter interface

Creating Your First Notebook

In Jupyter, click "New" (top right)

Select "Python 3 (ipykernel)"

A new notebook opens

In the first cell, type:

```
print("Hello, AIM101!")
```

Press `Shift + Enter` to run the cell

You should see: `Hello, AIM101!`

Congratulations! Your environment is ready.

Closing Jupyter

Save your notebook: `Ctrl + S` (or `Cmd + S` on Mac)

Close the browser tab

In the terminal, press `Ctrl + C` twice to stop Jupyter

Part 5: Environment Verification Checklist

Complete this checklist BEFORE the live session. Check each box only after successful verification.

Verification Commands

Run each command and verify the output:

Step	Command	Expected Output	✓
1	<code>conda activate mlenv</code>	Prompt shows <code>(mlenv)</code>	☐
2	<code>python --version</code>	<code>Python 3.12.x</code>	☐
3	<code>jupyter notebook</code>	Browser opens with Jupyter	☐

Step	Command	Expected Output	✓
4	Create new Python 3 notebook	Notebook opens	☐
5	Run <code>print("Hello")</code>	Shows <code>Hello</code>	☐

Final Test Script

Create a new notebook and run this code in a cell:

```
# Environment verification script
import sys

print("=" * 40)
print("ENVIRONMENT VERIFICATION")
print("=" * 40)
print(f"Python Version: {sys.version}")
print(f"Python Path: {sys.executable}")

# Check version
version = sys.version_info
if version.major == 3 and version.minor == 12:
    print("\n✓ SUCCESS: Python 3.12 is correctly installed!")
else:
    print(f"\n✗ WARNING: Expected Python 3.12, got {version.major}.{version.minor}")

print("=" * 40)
```

If you see "SUCCESS", you're ready for class!

Things to Ponder

Before class, think about these questions. We'll explore them during the session.

Why do programming languages need different data types?

- Why can't everything just be "data"?
- What problems might arise without type distinctions?

What makes Python easier to read than other languages?

- Have you seen code in other languages? What looked confusing?

Why might `"5" + "3"` give `"53"` instead of `8`?

- What rule is Python following?
- When might you WANT this behavior?

If `input()` always returns a string, why is that?

- How would Python know if you typed a number or text?

- What problems does this design prevent?

Why do we start counting at 0 instead of 1?

- This is called "zero-based indexing"
- What are the advantages and disadvantages?

Explore: Python's Official Tutorial

<https://docs.python.org/3.12/tutorial/introduction.html>

Sections 3.1 (Numbers) and 3.2 (Strings) cover what we'll learn in class.

Pre-Class Checklist

Before attending the live session, confirm ALL items:

Environment Setup

- ☐ Python 3.12 installed and verified
- ☐ Miniconda installed and verified
- ☐ `m1env` environment created with Python 3.12
- ☐ Jupyter Notebook installed in `m1env`
- ☐ Successfully ran `print("Hello")` in a notebook

Technical Readiness

- ☐ Laptop fully charged or plugged in
- ☐ Stable internet connection
- ☐ Browser works with Jupyter (Chrome/Firefox recommended)
- ☐ Know how to activate your environment

Troubleshooting Contacts

If you encounter issues:

First, check the troubleshooting tables in this document

Search the error message online

Contact course support: [Insert support contact]

Post in the course discussion forum

Quick Reference: Commands You'll Need

Environment Commands

```
conda activate mlenv      # Activate your environment
conda deactivate          # Deactivate environment
jupyter notebook          # Launch Jupyter
```

In Jupyter

- **Shift + Enter** — Run current cell and move to next
- **Ctrl + Enter** — Run current cell and stay
- **Esc + A** — Insert cell above
- **Esc + B** — Insert cell below
- **Esc + DD** — Delete current cell
- **Ctrl + S** — Save notebook

See You in Class!

Vishlesan i-Hub IIT Patna × Masai School